Perfil AgentSpeak (Jason/JaCaMo)

GUÍA DE DESARROLLO. VERSIÓN: 1.0 (IADK 1.0)

FECHA: 10/11/2026

IAg Development Kit

Generación de código para Jason, CArtAgO y Moise (JaCaMo 1.3)

Este documento explica cómo desarrollar aplicaciones Jason/JaCaMo con el perfil AgentSpeak del IAg Development Kit (IADK). El perfil adapta el editor de INGENIAS a JaCaMo: los agentes y sus planes se modelan con diagramas INGENIAS, y el generador AgentSpeak produce un proyecto JaCaMo 1.3 completo, con los programas AgentSpeak (Jason) de los agentes, los artefactos CArtAgO de su entorno (en Java o en Python), las organizaciones Moise y la aplicación .jcm que lo une todo.

La guía sigue un ejemplo, desde el proyecto Hello world del creador de proyectos JaCaMo hasta un agente que cuenta con un artefacto, alcanza un objetivo definido por una condición sobre sus creencias y desempeña un rol en una organización Moise. También describe cómo se traduce cada elemento del modelo, cómo se mantiene el modelo sincronizado con el código y cómo usar con los proyectos JaCaMo los agentes de IA del editor.

Se supone que el lector conoce el editor del IADK (manual del IADK, doc/IADKmanual-1.0-es.pdf) y los fundamentos de Jason, CArtAgO y Moise (https://jacamo-lang.github.io).

ÍNDICE

1. INTRODUCCIÓN 4

1.1 El perfil AgentSpeak 4

1.2 De INGENIAS a JaCaMo 5

2. PRIMEROS PASOS 6

2.1 Requisitos 6

2.2 Crear un proyecto 6

2.3 Generar y ejecutar el código 7

2.4 El ejemplo de esta guía 7

3. DESARROLLO DEL EJEMPLO PASO A PASO 9

4. AGENTES, CREENCIAS, OBJETIVOS Y PLANES 11

4.1 Agentes 11

4.2 Creencias y objetivos iniciales 11

4.3 Planes 12

5. CÓDIGO DE PLANES, TAREAS Y AGENTES 14

6. REGLAS DE UN PLAN: TASKS AND GOALS MODEL 15

6.1 Condición de satisfacción de un objetivo 15

6.2 Acciones de entorno y de organización de una regla 16

7. ENTORNO: ARTEFACTOS CARTAGO 17

7.1 Percepciones y señales: disparadores de las reglas 18

7.2 Acciones internas 19

7.3 Artefactos y acciones internas en Python 19

8. ORGANIZACIÓN: MOISE 20

9. MENSAJES ENTRE AGENTES 22

10. DESPLIEGUES 23

11. MANTENER SINCRONIZADOS EL MODELO Y EL CÓDIGO 24

12. AGENTES DE IA EN EL EDITOR 25

12.1 Usar la pestaña de un agente 25

12.2 Los agentes de una instalación 25

12.3 Gestionar los agentes 26
12.4 Paso a paso: Claude Code 27
12.5 Paso a paso: otros agentes de línea de órdenes (Codex, Gemini, Ollama...) 28
12.6 Paso a paso: API de Anthropic 29
12.7 Paso a paso: API compatibles con OpenAI (OpenAI, Ollama, LM Studio) 30
12.8 Agentes para un perfil 31
12.9 Peticiones útiles 31
12.10 Problemas frecuentes con los agentes de IA 32

13. PROBLEMAS FRECUENTES 33

1. INTRODUCCIÓN

JaCaMo es una plataforma de programación multiagente que combina tres tecnologías, una por cada dimensión de un sistema multiagente:

- **Jason**, para los agentes: agentes BDI programados en AgentSpeak, con creencias, objetivos y planes (reglas de la forma `+!objetivo : contexto <- cuerpo.`).
- **CArtAgO**, para el entorno: artefactos, agrupados en workspaces, que ofrecen operaciones a los agentes y propiedades observables que los agentes perciben como creencias; los artefactos también pueden emitir señales.
- **Moise**, para la organización: grupos y roles (especificación estructural), esquemas de objetivos y misiones (especificación funcional) y normas que obligan o permiten a los roles comprometerse con misiones (especificación normativa).

El perfil AgentSpeak del editor del IADK modela estas tres dimensiones con diagramas INGENIAS, y el **generador Jason/JaCaMo AgentSpeak** produce a partir de ellos un proyecto JaCaMo 1.3 estándar. El proyecto se ejecuta con Gradle (./gradlew run), que descarga JaCaMo la primera vez.

1.1 El perfil AgentSpeak

El perfil se elige con Profiles > AgentSpeak. Entonces el editor:

- ofrece en cada diagrama solo las entidades y relaciones que usa el generador (paleta, entradas *Insert* del menú contextual y relaciones que se ofrecen al relacionar dos elementos), y oculta las relaciones de especificaciones existentes que el perfil no usa (siguen en el modelo, y el perfil Full INGENIAS las vuelve a mostrar);
- muestra en los diálogos de edición solo los atributos que usa el generador (se conservan los valores de los demás);
- organiza el árbol de entidades según la organización: sus primeros nodos son las organizaciones, con sus roles, grupos, agentes miembros y esquemas;
- lista en el menú Modules los módulos de los proyectos JaCaMo (figura 1).

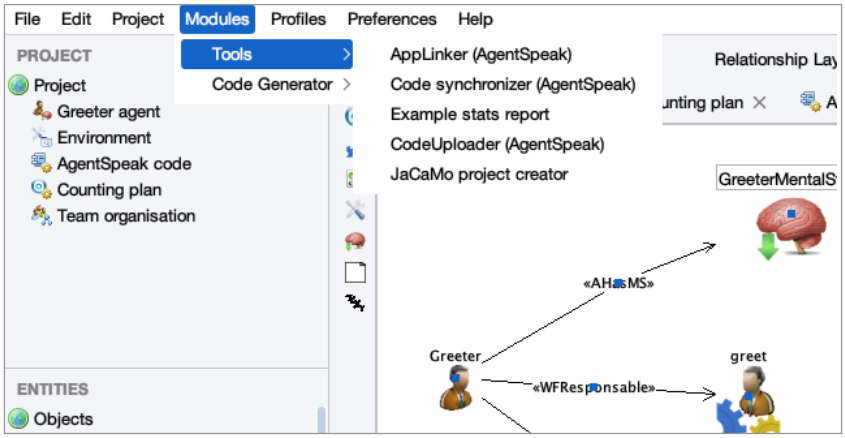


Figura 1: Las herramientas del perfil AgentSpeak (Modules > Tools)

Módulo (menú)	Para qué sirve
Jason/JaCaMo AgentSpeak generator (Code Generator)	Genera el proyecto JaCaMo de la especificación.
JaCaMo project creator (Tools)	Crea un proyecto a partir de un ejemplo (Hello world, Hello world with Python) o vacío, genera su código y lo abre.
CodeUploader (AgentSpeak) (Tools)	Sube a los componentes de código del modelo las reglas editadas en los ficheros .asl.
AppLinker (AgentSpeak) (Tools)	Actualiza los artefactos y las acciones internas del modelo a partir de su código (Java o Python).

Módulo (menú)	Para qué sirve
Code synchronizer (AgentSpeak) (Tools)	Ambas cosas (capítulo 11).
HTML Document generator (Code Generator)	Documentación HTML de la especificación.

1.2 De INGENIAS a JaCaMo

La tabla siguiente resume cómo se traducen los elementos del modelo. Los capítulos siguientes explican cada uno.

Diagrama	Elemento	JaCaMo
Agent Model	Agent	Agente Jason: src/agt/<agente>.asl y su entrada en el .jcm.
Agent Model	MentalState (AHasMS) con FrameFacts y Goals (AContainsME), GTPursues	Creencias iniciales (`count(5).`) y objetivos iniciales (`!start.`).
Agent Model	Plan del agente (WFResponsable) que satisface un Goal (GTSatisfies)	Reglas para el objetivo, `+!objetivo : contexto <- cuerpo.`, con el código de su componente de código.
Agent Model	Role (WFPlays)	El agente recibe el estado mental y los objetivos del rol, y lo desempeña en la organización.
Tasks and Goals Model	Task de un Plan (WFContainsTask)	Una regla del plan, refinada por sus relaciones (capítulo 6).
Tasks and Goals Model	SatisfactionBeliefs de GTSatisfies	Objetivo declarativo: reglas que terminan el objetivo cuando se cumplen sus creencias (sección 6.1).
Component Diagram	AslCodeComponent (CDUsesCode)	Código AgentSpeak de un plan, una tarea o un agente.
Environment Model	Workspace, EnvironmentApplication (EContainsArtifact)	Workspace y artefacto CArtAgO (src/env/tools/<Artefacto>.java).
Environment Model	Methods de una EnvironmentApplication; FrameFact (EHasObservableProperty)	Operaciones del artefacto; propiedades observables.
Environment Model	EPerceives, EPerceivesPolling, EPerceivesNotification	Focus del agente en el artefacto, las operaciones que usa y las percepciones y señales que disparan sus reglas.
Environment Model	InternalApplication	Biblioteca de acciones internas (src/agt/<biblioteca>/<acción>.java).
Organization Model	Organization, OrganizationGroup, Role, Scheme, Mission, Goal, WFParticipates	Organización Moise src/org/<organización>.xml, sus grupos y esquemas en el .jcm, y los roles de los agentes.
Interaction Model	UMLSendsMessage	Un mensaje (.send) del agente emisor.
Deploy Diagram	DeploymentPackage, unidades de despliegue	Instancias de los agentes en el .jcm, con sus creencias y objetivos iniciales.

2. PRIMEROS PASOS

2.1 Requisitos

- JDK 17 o posterior (el JDK completo) y conexión a Internet la primera vez que se ejecuta un proyecto: Gradle y las bibliotecas de JaCaMo 1.3.0 se descargan y quedan guardados.
- La distribución AgentSpeak del IADK (IADK-1.0-AgentSpeak-<fecha>.zip), o una distribución con todos los perfiles. Arranca el editor como explica el manual del IADK (IADK.bat, IADK.command o scripts/run-editor).
- Opcionalmente, Python 3, para artefactos y acciones internas escritos en Python (sección 7.3).

2.2 Crear un proyecto

1. Arranca el editor y elige Profiles > AgentSpeak.
2. Elige Modules > Tools > JaCaMo project creator (figura 2).
3. Elige la carpeta del proyecto (no debe contener ya una especificación) y el punto de partida: *Hello world*, un agente que saluda y usa un artefacto contador; *Hello world with Python*, lo mismo con el artefacto contador y una acción interna escritos en Python; o *Empty specification*.
4. Pulsa *Create*. El creador escribe la especificación (spec/specification.xml), genera el código del proyecto y abre la especificación en el editor.

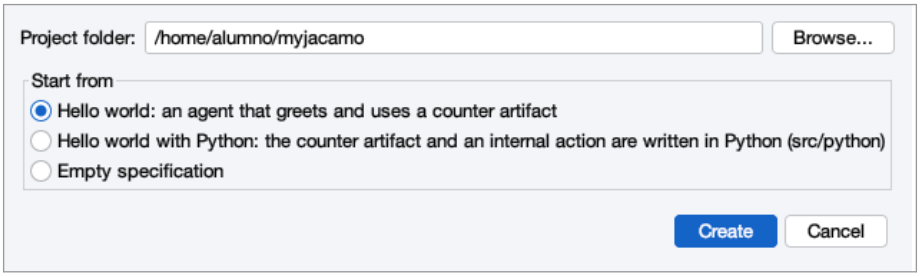


Figura 2: El creador de proyectos JaCaMo

El proyecto tiene esta estructura:

```
myjacamo/
  spec/specification.xml           el modelo
  myjacamo.jcm                     la aplicación JaCaMo (se reescribe en cada generación)
  src/agt/<agente>.asl             el código AgentSpeak de cada agente (se reescribe)
  src/env/tools/<Artefacto>.java   los artefactos CArtAgO (se escriben una vez)
  src/agt/<biblioteca>/<acc>.java  las acciones internas (se escriben una vez)
  src/org/<organización>.xml       las organizaciones Moise (se reescriben)
  src/python/                      los módulos Python de los artefactos (Python)
  build.gradle, settings.gradle, gradlew, gradlew.bat, logging.properties  (se escriben
una vez)
```

Los ficheros que se *reescriben* en cada generación no deben editarse a mano, salvo las regiones de los componentes de código de los ficheros .asl, que se suben al modelo (capítulo 11). Los ficheros que se *escriben una vez* pertenecen desde entonces al desarrollador: el generador nunca los sustituye, y lo dice en la pestaña Logs.

2.3 Generar y ejecutar el código

El código se genera con Modules > Code Generator > Jason/JaCaMo AgentSpeak generator > Generate. La pestaña Logs lista los ficheros escritos y los errores y avisos del modelo, con enlaces a los elementos a los que se refieren. El generador escribe el proyecto junto a la especificación (la carpeta que contiene su carpeta spec); las propiedades *JaCaMo project folder* y *JaCaMo application name* de Project > Properties permiten elegir otra carpeta y otro nombre para el .jcm (vacías: la carpeta de la especificación y su nombre; figura 3). El código también se puede generar desde la línea de órdenes, en la carpeta editor del IADK:

```
cd editor
java -cp "lib/ingeniaseditor.jar:lib/*:lib/commons-codec/*:ext/modagentspeak.jar" \
     ingenias.codeproc.AgentSpeakGenerator <proyecto>/spec/specification.xml
```

En Windows, el classpath se separa con ; en lugar de :.

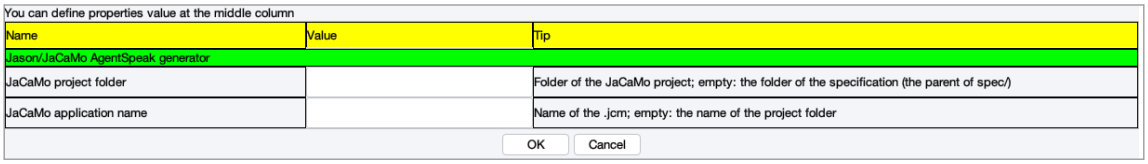


Figura 3: Propiedades del generador AgentSpeak (Project > Properties)

La aplicación se ejecuta en la carpeta del proyecto:

```
cd myjacamo
./gradlew run          (Windows: gradlew.bat run)
```

Lo que imprimen los agentes (.print) aparece en el terminal. Mientras la aplicación se ejecuta, JaCaMo ofrece inspectores web, cuyas direcciones se imprimen al arrancar: el inspector de la mente de los agentes (http://127.0.0.1:3272 por defecto) y los inspectores de CArtAgO y de Moise. La aplicación se detiene con Ctrl+C. Para usar la ventana de consola de JaCaMo en lugar del terminal, cambia el handler de logging.properties como indica su comentario.

2.4 El ejemplo de esta guía

Esta guía convierte el proyecto Hello world del creador en una pequeña aplicación, que muestran las figuras de los capítulos siguientes. El agente *greeter* saluda (plan greet, para su objetivo inicial start) y usa el artefacto *counter*, cuya operación inc incrementa la propiedad observable count. El ejemplo añade:

- un objetivo *reach_target*, que alcanza el plan *counting*, cuya tarea *step* incrementa el contador hasta llegar a 3 y entonces cree *enough*; el objetivo se satisface con la creencia *enough* (capítulos 4 y 6);
- el workspace *env* del artefacto (capítulo 7);
- una organización Moise, *counting_org*, con un grupo *team*, un rol *counter_role* que desempeña el greeter y un esquema *count_sch* cuya misión *m_count* tiene el objetivo reach_target; una norma obliga al rol a comprometerse con la misión (capítulo 8).

Al ejecutarse, el agente saluda, se compromete con su misión por la norma y cuenta hasta que se satisface el objetivo:

```
[Moise] OrgBoard counting_org created.
[Moise] scheme created: count_sch: count_sch using artifact SchemeBoard
[Moise] group created: team: team using artifact ora4mas.nopl.GroupBoard
[Cartago] artifact counter: tools.Counter() at env created.
[greeter] join workspace /main/env: done
[greeter] focus on counter: done
[greeter] Hello world!
[greeter] I am obliged to commit to m_count on count_sch... doing so
[greeter] The counter is now 1
[greeter] The counter is now 2
```

```
[greeter] The counter is now 3
```

La figura 4 muestra el editor con la especificación del ejemplo.

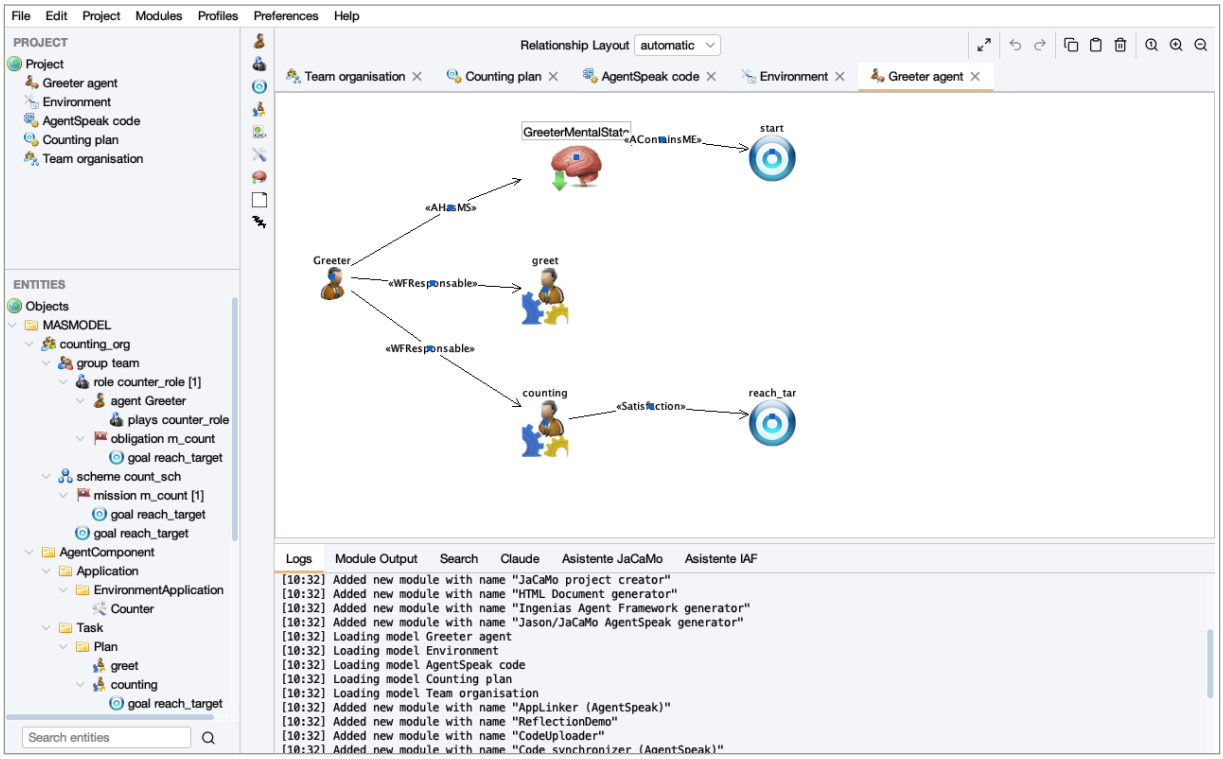


Figura 4: El editor con el perfil AgentSpeak y la especificación del ejemplo. El árbol de entidades empieza por la organización

3. DESARROLLO DEL EJEMPLO PASO A PASO

Este capítulo construye paso a paso el ejemplo de la sección 2.4 a partir del proyecto Hello world. Los capítulos siguientes explican cada elemento en detalle. Recuerda las operaciones básicas del editor (manual del IADK, capítulo "El editor gráfico"): una entidad se inserta con la paleta del diagrama o con las entradas *Insert* de su menú contextual; con un doble clic se abre su diálogo de edición, donde su id es su nombre; dos entidades se relacionan arrastrando desde el centro de la primera hasta la segunda, y el editor pregunta qué relación crear si hay varias posibles; una entidad existente se añade a otro diagrama con *Add to current graph* en la vista de entidades (o copiando y pegando).

Paso 1. Crear y ejecutar el proyecto Hello world. Crea el proyecto con el creador de proyectos JaCaMo (*Hello world*, sección 2.2). Su especificación tiene tres diagramas: *Greeter agent* (el agente, su estado mental con el objetivo start y su plan greet), *Environment* (el artefacto Counter con su operación inc y su propiedad observable count) y *AgentSpeak code* (el código del plan greet). Ejecútalo con ./gradlew run en la carpeta del proyecto: el agente imprime "Hello world!" y "The counter is now 1".

Paso 2. Un objetivo nuevo y su plan (Agent Model). Abre el diagrama *Greeter agent*. Inserta un Goal llamado reach_target y un Plan llamado counting. Relaciona Greeter con counting (elige WFResponsable: el agente es responsable del plan) y counting con reach_target (GTSatisfies: el plan se declara para el objetivo). El diagrama queda como el de la figura 5.

Paso 3. Las reglas del plan (Tasks and Goals Model). Pulsa el botón derecho sobre *Project* en la vista del proyecto, elige *Add TasksAndGoalsModel* y llama *Counting plan* al diagrama. Añádele los existentes Greeter, counting, reach_target y Counter (vista de entidades, *Add to current graph*). Inserta una Task llamada step y un FrameFact llamado enough. Relaciona counting con step (WFContainsTask: la tarea es una regla del plan), step con enough (GTCreates: la regla añade la creencia) y step con Counter (WFUses). Haz doble clic sobre la WFUses y marca la operación inc en sus Operations (figura 11). El diagrama queda como el de la figura 9.

Paso 4. El código de la tarea (Component Diagram). Abre *AgentSpeak code*, añade la tarea step, inserta un AslCodeComponent llamado step_code y relaciona step con step_code (CDUsesCode). Haz doble clic sobre step_code y escribe en su Code el cuerpo de la regla (figura 8):

```
inc;
?count(N);
.print("The counter is now ", N);
if (N >= 3) { +enough };
!reach_target
```

Paso 5. La condición de satisfacción del objetivo. Haz doble clic sobre la GTSatisfies de counting a reach_target (diagrama *Greeter agent*) y marca la creencia enough en sus SatisfactionBeliefs (figura 10): el objetivo se satisface cuando el agente cree enough.

Paso 6. El workspace del artefacto (Environment Model). Abre *Environment*, inserta un Workspace llamado env y relaciónalo con Counter (EContainsArtifact). El workspace por defecto de JaCaMo también se llama env, así que la aplicación generada no cambia, pero el modelo dice ahora dónde está el artefacto (figura 12).

Paso 7. La organización (Organization Model). Añade al proyecto un OrganizationModel (*Add OrganizationModel*) llamado *Team organisation*. Inserta una Organization llamada counting_org, un OrganizationGroup llamado team, un Role llamado counter_role, un Scheme llamado count_sch y una Mission llamada m_count, y añade los existentes reach_target y Greeter. Relaciónalos (figura 15):

1. counting_org con team (OHasGroup) y con count_sch (OHasWF);
2. team con counter_role (OHasMember: el grupo tiene el rol);
3. count_sch con m_count (WFDecomposesWF: la misión del esquema), y count_sch y m_count con reach_target (GTSatisfies: el objetivo del esquema y de la misión);

4. counter_role con m_count (WFParticipates: la norma); en su diálogo, Deontic es obligation (figura 16);
5. Greeter con counter_role (WFPlays: el agente desempeña el rol).

Paso 8. Generar y ejecutar. Guarda la especificación (File > Save), genera el código (Modules > Code Generator > Jason/JaCaMo AgentSpeak generator > Generate) y revisa la pestaña Logs. El generador escribe src/org/counting_org.xml, reescribe greeter.asl y el .jcm, y conserva Counter.java, que ya tiene el código de inc. Vuelve a ejecutar la aplicación: el agente saluda, se compromete con su misión y cuenta hasta que se satisface el objetivo, como muestra la sección 2.4.

Paso 9. Cambiar el código y subirlo al modelo. Edita src/agt/greeter.asl y cambia, en el bloque de la tarea step, la condición `N >= 3` por `N >= 5`. Ejecuta la aplicación: el agente cuenta hasta 5. Después ejecuta Modules > Tools > Code synchronizer (AgentSpeak): muestra el cambio del código de la tarea step (figura 19). Pulsa *Apply selected* y guarda la especificación. Al volver a generar el código se conserva el cambio, porque ya está en el modelo.

4. AGENTES, CREENCIAS, OBJETIVOS Y PLANES

El Agent Model describe cada agente: su estado mental (creencias y objetivos iniciales), los planes de los que es responsable y los objetivos que satisfacen, y los roles que desempeña (figura 5).

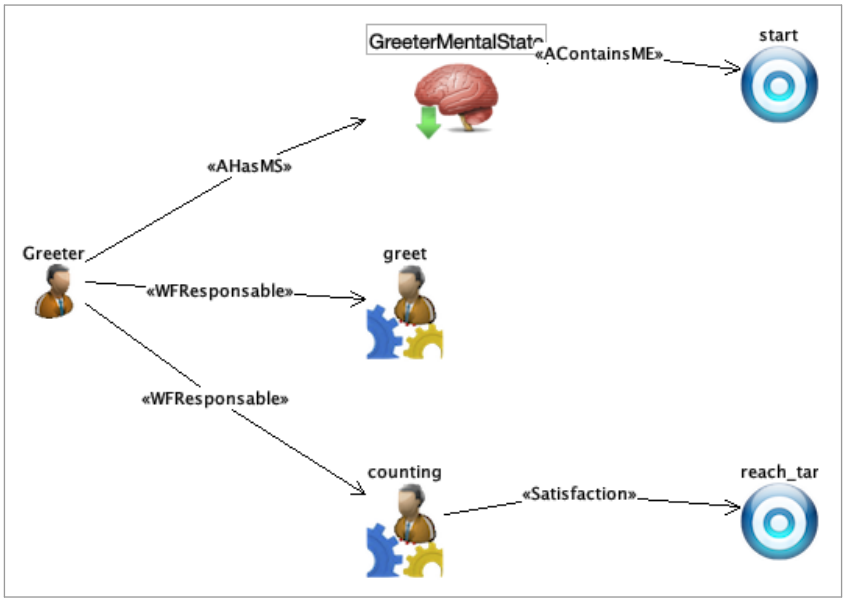


Figura 5: Agent Model del ejemplo

4.1 Agentes

Cada Agent es un agente Jason, con su programa AgentSpeak en src/agt/<agente>.asl y una entrada en el .jcm. Los nombres se convierten en átomos AgentSpeak válidos: el agente Greeter es greeter, y greeter.asl es su programa. El atributo aslFile del componente de código del agente puede indicar otro fichero ("greeter" para src/agt/greeter.asl).

Sin despliegues (capítulo 10), el .jcm arranca un agente de cada tipo, que se une a los workspaces que usa y hace focus en los artefactos que percibe:

```
agent greeter : greeter.asl {
    roles: counter_role in counting_org.team
    join: env
    focus: env.counter
}
```

4.2 Creencias y objetivos iniciales

El MentalState del agente (AHasMS) contiene sus creencias y objetivos iniciales (AContainsME):

- Un **FrameFact** es una creencia: el término de sus slots con sus valores. Un FrameFact count con un slot cuyo Value es 5 es la creencia `count(5)`. Para dar un valor, edita el FrameFact, abre su slot (campo Slots) y escribe su Value. Da valor a todos los slots (figura 6) y escribe las cadenas entre comillas dobles ("hello").
- Un **Goal** es un objetivo inicial: `!start`. Los objetivos que persigue el agente (GTPursues) también son objetivos iniciales.



Figura 6: Diálogo de edición de un slot: la creencia count(5) del agente

Un agente recibe también el estado mental y los objetivos de los roles que desempeña (WFPlays) y de los roles que estos extienden (ARoleInheritance).

4.3 Planes

El agente es responsable de sus Plans (WFResponsable). Un Plan declarado para satisfacer un Goal (GTSatisfies) es un conjunto de reglas para ese objetivo: `+!objetivo : contexto <- cuerpo`. Las reglas salen de:

- el componente de código del plan (capítulo 5), cuando el plan no tiene tareas: un plan sin tareas es una única regla;
- las tareas del plan, una regla cada una, cuando las tiene (capítulo 6).

Un plan (o tarea) del agente que no satisface ningún objetivo ni consume ningún evento es una intención inicial: el generador añade al agente el objetivo !<plan>, si sus reglas se disparan con él. En el ejemplo, el plan greet tiene reglas completas para +!start, el objetivo inicial del agente, y el plan counting satisface reach_target.

El .asl del agente tiene una sección por cada tipo de elemento del modelo: creencias iniciales, objetivos iniciales, objetivos satisfechos por creencias, planes, otras reglas del agente, los artefactos en los que hace focus, las percepciones y señales que disparan sus reglas, las operaciones que usa, los roles que desempeña, los protocolos de interacción y los mensajes que envía. Los comentarios de cada sección indican de qué elementos del modelo sale. Este es el programa del greeter del ejemplo:

```
!start.

//#start_generated:reach_target_satisfied <--- GENERATED FROM THE MODEL
@reach_target_satisfied
+!reach_target : enough <- true.
@reach_target_achieved_enough
+enough : .desire(reach_target) <- .succeed_goal(reach_target).
//#end_generated:reach_target_satisfied

//#start_node:greet <--- DO NOT REMOVE THIS
+!start : true <-
    .print("Hello world!");
    inc;
    ?count(N);
    .print("The counter is now ", N).
//#end_node:greet <--- DO NOT REMOVE THIS

//#start_node:step <--- DO NOT REMOVE THIS
@step
+!reach_target : true <-
    inc;
    ?count(N);
    .print("The counter is now ", N);
    if (N >= 3) { +enough };
    !reach_target.
//#end_node:step <--- DO NOT REMOVE THIS
```

```
{ include("$jacamo/templates/common-cartago.asl") }
{ include("$jacamo/templates/common-moise.asl") }
{ include("$moise/asl/org-obedient.asl") }
```

5. CÓDIGO DE PLANES, TAREAS Y AGENTES

El código AgentSpeak del modelo está en componentes de código: un AslCodeComponent unido con CDUsesCode a un plan, una tarea o un agente en un Component Diagram (figura 7). Su Language debe ser AgentSpeak, el valor por defecto de AslCodeComponent: el generador solo escribe código AgentSpeak en los ficheros .asl, y da un error con un componente en otro lenguaje. INGENIASCodeComponent solo se usa para las aplicaciones.

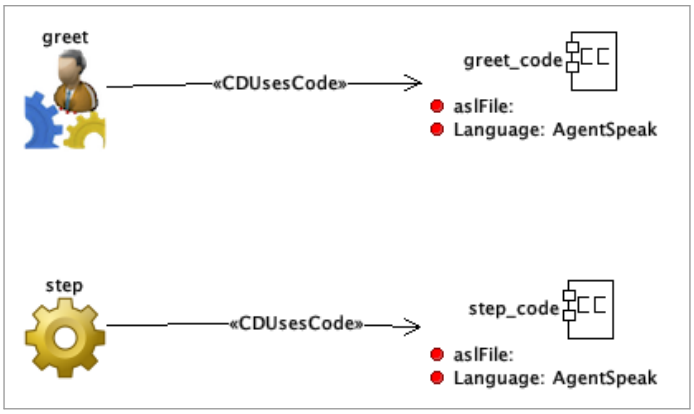


Figura 7: Component Diagram del ejemplo: el código del plan greet y de la tarea step

Un componente contiene o bien:

- **reglas completas**, como `+!start : true <- .print("Hello world!").`, que se escriben tal cual (el plan greet del ejemplo); o
- **solo el cuerpo de la regla**, como `.print("Hello world!"); inc`, que el generador completa con el disparador (el objetivo que satisface la tarea o el plan, o el objetivo de su plan; sin objetivos, el nombre del plan o de la tarea), el contexto (las creencias que consume, o true) y una etiqueta con el nombre de la tarea: `@step +!reach_target : true <- cuerpo.` (la tarea step del ejemplo).

Un componente de código del agente contiene otras reglas del agente (sección "Other rules of the agent" del .asl). En el .asl, el código de cada componente está entre las marcas //#start_node:<elemento> y //#end_node:<elemento>. Las reglas se pueden editar allí y subir al modelo con el CodeUploader (AgentSpeak) (capítulo 11); lo que se escriba fuera de las marcas se pierde al volver a generar el código.



Figura 8: Diálogo de edición del AslCodeComponent de la tarea step

6. REGLAS DE UN PLAN: TASKS AND GOALS MODEL

En el Tasks and Goals Model un Plan contiene sus reglas, las Tasks (WFContainsTask). Cada Task es una regla `+!objetivo : contexto <- cuerpo.` para el objetivo de su plan, o para el objetivo que satisface la propia tarea (GTSatisfies). Las relaciones de la tarea refinan su regla (figura 9):

Relación de la Task	En la regla
WFConsumes FrameFact	Una creencia del contexto de la regla.
WFConsumes ApplicationEventSlots	La percepción o señal que dispara la regla: `+tick(N) : true <- ...` (sección 7.1).
GTCreates / GTDestroys / GTModifies FrameFact	Creencias que la regla añade, elimina o actualiza.
GTCreates / WFCancels Conversation	Mensajes que inicia o cancela.
WFDecomposes Task	Una subtarea, que se llama con !subtarea; es una regla del mismo agente.
WFUses, GTCreates, GTDestroys, WFCancels con entidades del entorno y de la organización	Acciones de la regla (sección 6.2).

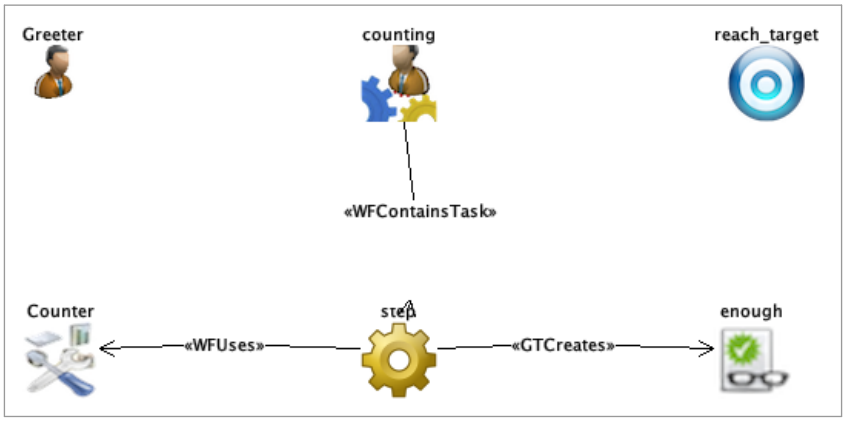


Figura 9: Tasks and Goals Model del ejemplo: la tarea step del plan counting crea la creencia enough y usa la operación inc del contador

En el ejemplo, la tarea *step* del plan *counting* tiene el cuerpo `inc; ?count(N); .print("The counter is now ", N); if (N >= 3) { +enough }; !reach_target`: incrementa el contador, lee su propiedad observable count, añade la creencia enough cuando el contador llega a 3 y vuelve a perseguir el objetivo. El objetivo termina cuando se satisface (sección siguiente).

6.1 Condición de satisfacción de un objetivo

Las SatisfactionBeliefs de una GTSatisfies de un plan o una tarea son creencias (FrameFacts) de su agente: el objetivo se satisface cuando se cumplen todas. Su diálogo de edición (figura 10) lista, para marcarlas, las creencias del agente: las de su MentalState y de sus roles, las propiedades observables de los artefactos que percibe y las creencias que sus reglas añaden, cambian, eliminan o consultan. Un FrameFact es el término de sus slots, `count(3)`, con _ para un slot sin valor.

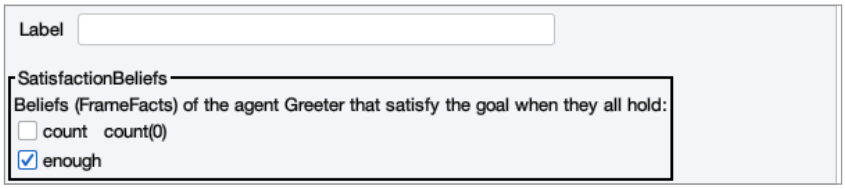


Figura 10: Las SatisfactionBeliefs de la GTSatisfies del plan counting: el objetivo reach_target se satisface con la creencia enough

El generador escribe, antes de los planes del agente, un patrón de objetivo declarativo:

```
@reach_target_satisfied
+!reach_target : enough <- true.
@reach_target_achieved_enough
+enough : .desire(reach_target) <- .succeed_goal(reach_target).
```

La primera regla hace que el objetivo se cumpla sin ejecutar ningún plan mientras se cumplen sus creencias; la segunda termina la intención del objetivo cuando pasan a cumplirse.

Las condiciones de varios planes para el mismo objetivo son alternativas. Estas reglas se generan desde el modelo, entre las marcas //#start_generated y //#end_generated: se cambian en el modelo, no en el .asl.

6.2 Acciones de entorno y de organización de una regla

En el Tasks and Goals Model una Task también se puede unir a workspaces, artefactos, bibliotecas de acciones internas, roles, grupos, esquemas y misiones. Si la tarea no tiene componente de código, el generador escribe las acciones correspondientes en el cuerpo de su regla (tras un mensaje TODO); si tiene código, avisa de las acciones que su código no hace.

Relación de la Task	Acciones de la regla
GTCreates Workspace	`createWorkspace("lab"); joinWorkspace("lab", _);`, makeArtifact para cada artefacto del workspace (EContainsArtifact) y focus en los que percibe el agente. Un workspace creado por una regla no está en el .jcm (ni sus artefactos).
WFUses EnvironmentApplication / InternalApplication	Llama a las operaciones marcadas en las Operations de la relación: `inc`, `util.show(X)`. El agente debe hacer focus en el artefacto (EPerceives).
WFUses Role	`adoptRole(worker)[artifact_name(team), wsp(shop)]`, en el grupo que usa la tarea, o en el grupo que tiene el rol.
WFUses OrganizationGroup	El grupo donde adopta sus roles; hace focus en él (lookupArtifact, focus).
WFUses Scheme	Hace focus en el esquema.
WFUses Mission	`commitMission(m)[artifact_name(scheme), wsp(org)]`.
WFCancels Role / Mission	leaveRole / leaveMission.
WFCancels OrganizationGroup	Abandona todos sus roles en el grupo.
GTDestroys OrganizationGroup	`destroy[artifact_name(team), wsp(shop)]`.
WFDecomposes Task	`!subtarea`.

El agente se une en el .jcm al workspace de las organizaciones que usa (el id de la organización). Moise comprueba sus normas: un agente no puede abandonar una misión cuyos objetivos no se han alcanzado, ni un rol que el grupo necesita para los esquemas de los que es responsable.

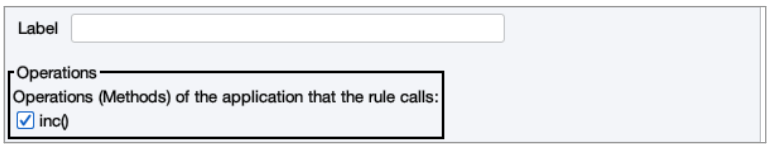


Figura 11: Las Operations de la WFUses de la tarea step al artefacto counter

7. ENTORNO: ARTEFACTOS CARTAGO

El Environment Model describe el entorno CArtAgO (figura 12):

Elemento	Traducción
Workspace	Un workspace CArtAgO (bloque workspace del .jcm). Los artefactos que no están en ningún workspace van al workspace por defecto, env.
EnvironmentApplication	Un artefacto, src/env/tools/<Artefacto>.java, creado en su workspace (EContainsArtifact : Workspace -> EnvironmentApplication).
Methods de la EnvironmentApplication	Sus operaciones (@OPERATION), con sus Parameters y su Result; un Result distinto de void es un parámetro de salida (OpFeedbackParam).
FrameFact (EHasObservableProperty)	Una propiedad observable del artefacto (defineObsProperty en init).
EPerceives, EPerceivesPolling, EPerceivesNotification : Agent -> EnvironmentApplication	El agente se une al workspace del artefacto y hace focus en él. Las Operations de la relación son las operaciones que usa el agente, y sus Events las percepciones y señales que disparan sus reglas.
EJoinsWorkspace : Agent -> Workspace	El agente se une al workspace.
InternalApplication	Una biblioteca de acciones internas: cada Method es una clase src/agt/<biblioteca>/<método>.java, que se llama como biblioteca.método(...).

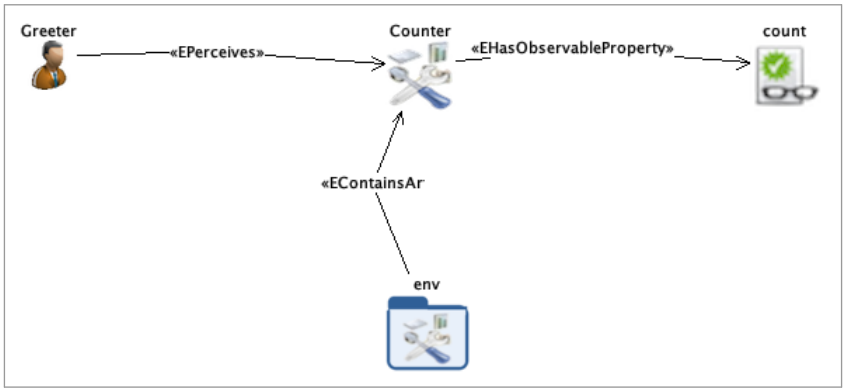


Figura 12: Environment Model del ejemplo: el artefacto counter, su propiedad observable count y su workspace env

El diálogo de edición de la EnvironmentApplication (figura 13) tiene sus Methods; cada Method tiene un Name, sus Parameters (Name y Type; Type out para un parámetro de salida) y su Result. El diálogo de la EPerceives del agente (figura 14) lista los Methods de la aplicación, y los marcados son las operaciones que usa el agente.



Figura 13: Diálogo de edición del artefacto counter (EnvironmentApplication) con su operación inc



Figura 14: La EPerceives del greeter: las operaciones que usa y los eventos que disparan sus reglas

La clase Java de un artefacto nuevo se genera una vez, con una operación por cada Method, la definición de sus propiedades observables y la lista de las señales que debe emitir; el desarrollador escribe el código de las operaciones. En el ejemplo, la operación inc incrementa la propiedad observable count:

```
package tools;
import cartago.*;

public class Counter extends Artifact {
    void init() {
        defineObsProperty("count", 0);
    }

    @OPERATION
    void inc() {
        ObsProperty count = getObsProperty("count");
        count.updateValue(count.intValue() + 1);
    }
}
```

Las propiedades observables de los artefactos en los que el agente hace focus son creencias del agente (count(N) en el ejemplo), y sus operaciones son acciones de sus reglas (inc). Cuando se escriben operaciones o propiedades observables nuevas en la clase, el AppLinker (AgentSpeak) las añade al modelo (capítulo 11).

7.1 Percepciones y señales: disparadores de las reglas

En el perfil AgentSpeak, un ApplicationEventSlots es el disparador de reglas de los agentes: una percepción de una propiedad observable o una señal de un artefacto. La relación del agente con el artefacto los lista en sus Events (en su diálogo de edición se pueden marcar o crear varios, porque una operación de CArtAgO puede cambiar varias propiedades observables y emitir varias señales), y su tipo indica qué son:

- **EPerceivesPolling**: percepciones de propiedades observables: el evento cuyo id empieza por el nombre de la propiedad (count para count_changed, o la única que haya) dispara `+count(Valor)`.

- **EPerceivesNotification**: señales que el artefacto emite con signal("tick", n): `+tick(N)`, siendo los slots del evento los argumentos de la señal.
- **EPerceives**: percepciones (los eventos cuyo id empieza por el nombre de una propiedad observable) y señales (los demás).

La tarea del agente que consume el evento (WFConsumes, Tasks and Goals Model) se dispara con él: `@onTick +tick(N) : true <- cuerpo`. En el Agent Model, la relación de un agente (o rol) con un ApplicationEventSlots (AExecutes) es una composición, dibujada con un rombo relleno en el agente: el disparador y sus reglas forman parte del código del agente. Un evento puede tener también su propio componente de código (CDUsesCode), cuyas reglas se generan para los agentes que lo perciben o reaccionan a él.

El .asl lista las percepciones y señales del agente y la tarea que atiende cada una, el esqueleto Java de un artefacto nuevo lista las señales que debe emitir, y el generador avisa de las señales que no atiende ninguna tarea y de las que el código del artefacto nunca emite.

7.2 Acciones internas

Una InternalApplication es una biblioteca de acciones internas de Jason: cada uno de sus Methods es una clase src/agt/<biblioteca>/<método>.java, generada una vez, que los agentes llaman como `biblioteca.método(Args)`. Sus Parameters y su Result son los argumentos de la acción; un Result distinto de void es un argumento de salida, que unifica la acción.

7.3 Artefactos y acciones internas en Python

Un INGENIASComponent con Language Python que realiza (UMLRealizes, Component Diagram) una EnvironmentApplication o una InternalApplication indica que sus operaciones son funciones de un módulo Python de src/python: el File indicado en los Files del componente (un FileSpecMapping), o <componente>.py. El generador escribe entonces:

```
src/python/<módulo>.py            una función por cada operación (se escribe una vez)
src/env/tools/<Artefacto>.java    el artefacto como pasarela al módulo
src/agt/<biblioteca>/<op>.java    cada acción interna como una llamada a él
src/java/pygateway               el soporte de las pasarelas (Java)
src/python/jacamo_gateway.py, jacamo_api.py     el soporte de las pasarelas (Python)
```

Las pasarelas y el soporte se reescriben cada vez; una clase escrita a mano nunca se sustituye.

- Cada operación es una función con sus parámetros de entrada (los Parameters de su Method); un parámetro con Type out, o su Result, es una salida, cuyo valor es lo que devuelve la función (una tupla si hay varias). Una excepción de Python hace fallar la operación o la acción interna.
- Cada artefacto ejecuta su propio proceso Python, así que las variables del módulo son el estado del artefacto. `jacamo_api.update(nombre, *valores)` actualiza (o define) sus propiedades observables y `jacamo_api.signal(nombre, *valores)` envía una señal a los agentes que hacen focus en él. Las acciones internas de una biblioteca comparten un proceso.
- Los agentes pueden llamar a cualquier función del módulo, esté o no modelada: `pycall("f", [Args], Resultado)` en el artefacto, `<biblioteca>.pycall(...)` como acción interna. Listas, números, cadenas y booleanos van en los dos sentidos; un diccionario llega como una lista de pares.
- El AppLinker y el Code synchronizer (AgentSpeak) leen los módulos: las funciones nuevas pasan a ser operaciones, y las propiedades actualizadas con jacamo_api.update, propiedades observables.
- Python 3 debe estar instalado: python3 (python en Windows), o la orden de la variable de entorno JACAMO_PYTHON. Lo que imprime el módulo aparece en el terminal de la aplicación.

El ejemplo *Hello world with Python* del creador de proyectos JaCaMo implementa en Python el contador y una acción interna.

8. ORGANIZACIÓN: MOISE

La organización Moise se modela en el Organization Model (figura 15):

Especificación	Elementos del Organization Model
Estructural	Organization (el grupo raíz), con sus grupos (OHasGroup) y sus propios roles y agentes (OHasMember : Organization -> Role / Agent); OrganizationGroup con sus roles (OHasMember), cada uno con la cardinalidad del rol (1, 0..1, `1..*`, 2..5...); Role (ARoleInheritance : Role -> Role, el rol que extiende; ORoleDependency : Role -> Role, un enlace Moise: conocimiento, comunicación o autoridad, con su alcance, si vale en los subgrupos y si es en los dos sentidos).
Funcional	Scheme de la Organization (OHasWF); sus Missions (WFDecomposesWF : Scheme -> Mission, con la cardinalidad de la misión); los objetivos del esquema y de las misiones (GTSatisfies), con MinAgents (cuántos agentes comprometidos deben alcanzarlo; por defecto, todos); los planes de los objetivos (GTDecomposesAND: subobjetivos en secuencia o en paralelo, en el orden en que se crearon las relaciones; GTDecomposesOR: elección).
Normativa	WFParticipates : Role -> Mission, una norma: obligación o permiso, con su condición y su restricción de tiempo (figura 16).

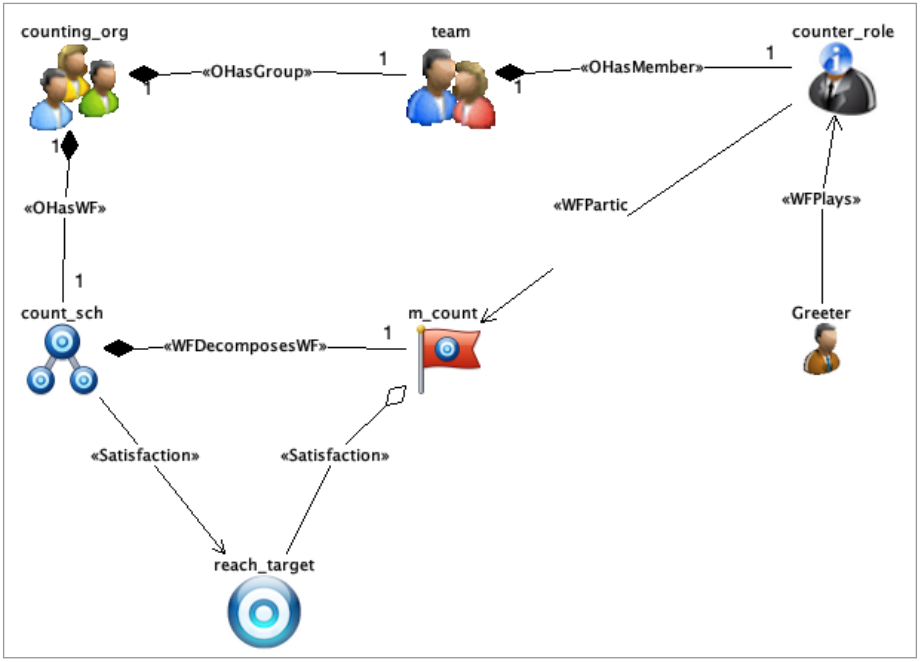


Figura 15: Organization Model del ejemplo



Figura 16: La norma del ejemplo: obligación de counter_role de comprometerse con m_count

Cada Organization se genera como src/org/<organización>.xml, una especificación Moise. Para el ejemplo:

```
<structural-specification>
  <role-definitions> <role id="counter_role"/> </role-definitions>
  <group-specification id="counting_org">
    <subgroups>
      <group-specification id="team" min="1" max="1">
        <roles> <role id="counter_role" min="1" max="1"/> </roles>
```

```
      </group-specification>
    </subgroups>
  </group-specification>
</structural-specification>
<functional-specification>
  <scheme id="count_sch">
    <goal id="reach_target"/>
    <mission id="m_count" min="1" max="1"> <goal id="reach_target"/> </mission>
  </scheme>
</functional-specification>
<normative-specification>
  <norm id="n1" type="obligation" role="counter_role" mission="m_count"/>
</normative-specification>
```

y su bloque del .jcm, con las instancias de sus grupos, los grupos responsables de los esquemas y los esquemas:

```
organisation counting_org : counting_org.xml {
    group counting_org_group : counting_org {
        group team : team {
            responsible-for: count_sch
        }
    }
    scheme count_sch : count_sch
}
```

Un agente desempeña sus roles (WFPlays) en los grupos que los tienen (el grupo raíz, para los roles de la Organization), o solo en los grupos de los que es miembro (OHasMember : grupo -> agente; un miembro de la Organization los desempeña en el grupo raíz y en todos sus grupos): `roles: counter_role in counting_org.team`. Si sus roles tienen normas, su programa incluye los planes obedientes de Moise (org-obedient.asl): se compromete con las misiones a las que está obligado y alcanza sus objetivos con sus planes para ellos (GTSatisfies). En el ejemplo, el greeter se compromete con m_count y alcanza reach_target con su plan counting.

El generador avisa de lo que Moise no admite: un esquema o una misión sin objetivos, una norma sobre un esquema, objetivos que no forman un único árbol, un objetivo repetido en un plan, ciclos de extensión de roles...

En el árbol de entidades, las organizaciones son los primeros nodos, con sus roles, grupos (con sus roles), agentes miembros y esquemas (con sus misiones y objetivos), y los agentes que desempeñan cada rol. Las entidades de la organización se pueden desplegar para ver sus relaciones (grupo, rol, extiende, esquema, misión, obligación...).

9. MENSAJES ENTRE AGENTES

Un mensaje entre agentes es una UMLSendsMessage del Interaction Model, del agente emisor al receptor. Su SpeechAct es la performativa (tell, untell, achieve, unachieve, askOne, askAll, tellHow, untellHow, askHow) y su Message el contenido. El .asl del emisor lista sus mensajes en la sección "Messages to other agents", listos para usarlos en sus reglas:

```
// .send(reporter, tell, count_reached(3)).
```

10. DESPLIEGUES

Sin despliegues, el .jcm arranca un agente de cada tipo. El Deploy Diagram describe cuántos agentes de cada tipo se arrancan y con qué estado inicial, como en el IAF (figura 17): cuando hay DeploymentPackages, los agentes del .jcm son los de sus unidades de despliegue (unidas al paquete con DefinesDeployment, o listadas en su AgentsDeployed). Cada unidad arranca NumberInstances agentes de su AgentTypeDeployed:

- un **DeploymentUnitByType** los arranca con el estado inicial de su tipo de agente;
- un **DeploymentUnitByTypeEnumInitMS** añade las creencias de su InitialState (figura 18): cada MentalInstanceSpecification es una instancia de un FrameFact (su InstanceType) con valores para sus slots (sus SlotsValues; un slot sin valor toma el Value del slot del FrameFact);
- un **DeploymentUnitByTypeMSEntity** añade las creencias y objetivos de un MentalState.



Figura 17: Un despliegue del ejemplo: el paquete two_counters con la unidad greeters



Figura 18: La unidad greeters: dos instancias de Greeter, con una creencia inicial target cuyo slot vale 5

Para el despliegue de las figuras, el agente del .jcm es:

```
agent greeter : greeter.asl {
    instances: 2
    beliefs: target(5)
    roles: counter_role in counting_org.team
    join: env
    focus: env.counter
}
```

Los tipos de agente que no despliega ninguna unidad no se arrancan, con un aviso. En el ejemplo, dos greeters que desempeñan counter_role necesitan que la cardinalidad del rol en el grupo team los admita (1..2, en la OHasMember de team a counter_role); si no, Moise rechaza el segundo.

11. MANTENER SINCRONIZADOS EL MODELO Y EL CÓDIGO

Durante el desarrollo se edita directamente el código del proyecto: reglas de los ficheros .asl, operaciones de los artefactos, funciones de los módulos Python. Tres módulos del perfil AgentSpeak suben esos cambios al modelo, para que la siguiente generación no los pierda. Primero muestran las diferencias (figura 19) y solo se aplican las que se dejan seleccionadas; después hay que guardar la especificación.

Módulo (Modules > Tools)	Qué actualiza
CodeUploader (AgentSpeak)	Los componentes de código de los planes, tareas y agentes, a partir de las reglas entre las marcas //#start_node y //#end_node de los ficheros .asl.
AppLinker (AgentSpeak)	Las EnvironmentApplications e InternalApplications, a partir de las operaciones (@OPERATION) y propiedades observables (defineObsProperty) de los artefactos y de las acciones internas, en Java o en los módulos Python de src/python. Crea las que solo existen en el código.
Code synchronizer (AgentSpeak)	Ambas cosas.

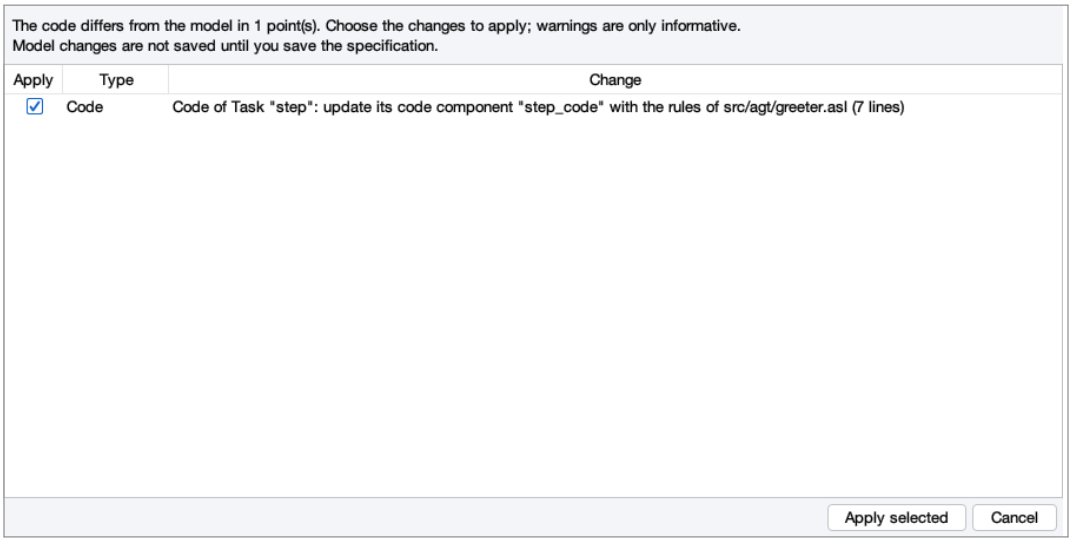


Figura 19: El Code synchronizer (AgentSpeak) tras cambiar la condición de la tarea step en greeter.asl

Un ciclo típico es:

1. Generar el código y ejecutar la aplicación.
2. Cambiar las reglas de un plan en su .asl, entre sus marcas, y las operaciones de los artefactos en sus clases; volver a ejecutarla.
3. Cuando funcione, ejecutar Modules > Tools > Code synchronizer (AgentSpeak), revisar los cambios y pulsar *Apply selected*.
4. Guardar la especificación y volver a generar el código: el .asl conserva los cambios, que ahora salen del modelo.

Las reglas escritas fuera de las marcas del .asl, y los cambios en las reglas generadas desde el modelo (//#start_generated ... //#end_generated), se pierden al generar el código: hay que cambiar el modelo.

Una especificación hecha con una versión anterior del perfil, en la que las operaciones eran ApplicationEventSlots (EProvidesOperation, AExecutes), se convierte al cargarla con el perfil AgentSpeak: las operaciones pasan a ser Methods, marcados en las Operations de las EPerceives de los agentes que las usaban, y la pestaña Logs dice qué ha cambiado. Guarda la especificación para conservar la conversión.

12. AGENTES DE IA EN EL EDITOR

Cada usuario del IADK puede añadir al editor uno o varios agentes de IA; cada agente tiene su propia pestaña junto a Logs, Module Output y Search (figura 20). Los agentes reciben como contexto con qué trabaja el usuario (el perfil activo, o uno fijo) y, opcionalmente, la especificación abierta y las carpetas de su proyecto. Pueden ayudar a completar las descripciones del modelo, comprobar su coherencia, escribir o revisar el código generado y el escrito a mano, y explicar conceptos de INGENIAS, el IAF, JADE, Jason, CArtAgO, Moise o JaCaMo.

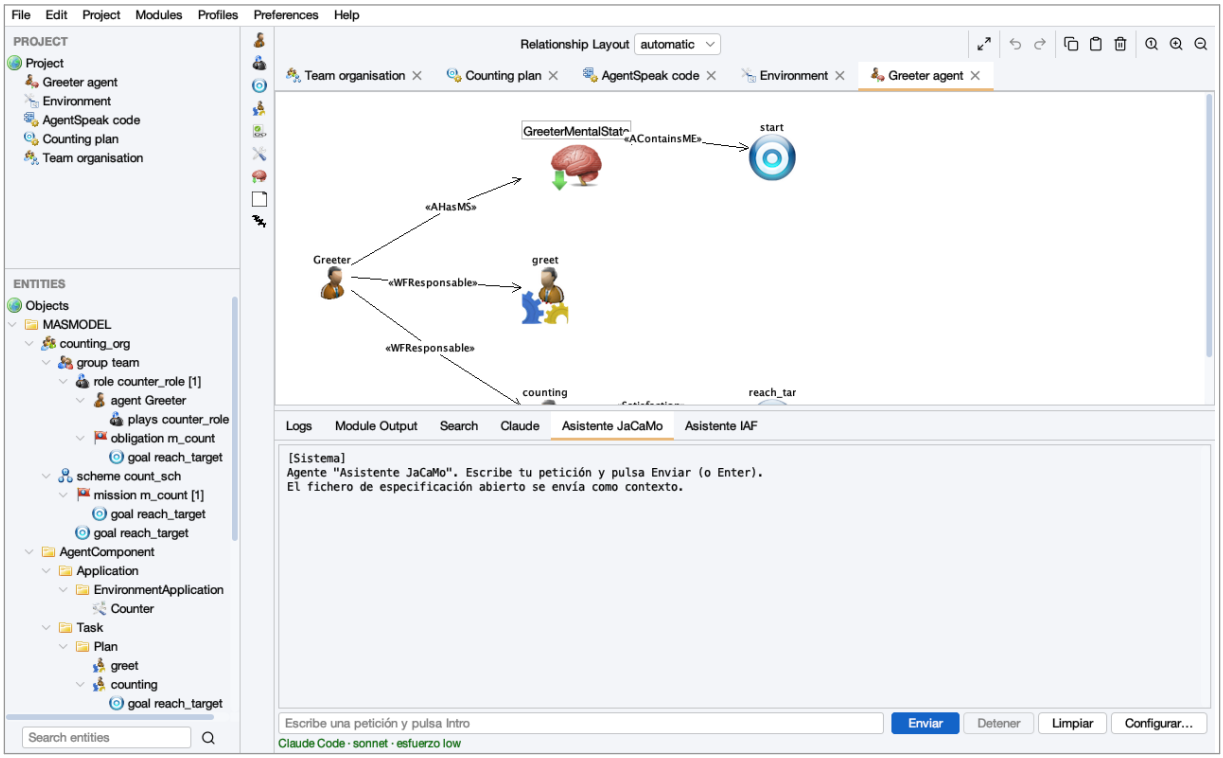


Figura 20: La pestaña del agente de IA "Asistente JaCaMo" en el editor

12.1 Usar la pestaña de un agente

Escribe una petición en el campo del pie de la pestaña y pulsa Intro o *Enviar*. La respuesta aparece a medida que se genera, y *Detener* la interrumpe. *Limpiar* empieza una conversación nueva y *Configurar...* abre la configuración de los agentes. La línea de estado indica qué programa o servicio usa el agente, y avisa si no lo encuentra o si falta la clave de API.

Si el agente tiene marcada la opción *Enviar como contexto la especificación abierta*, la especificación abierta se envía con la petición (el fichero guardado: guarda la especificación antes de preguntar por cambios recientes), junto con las carpetas de su proyecto (gensrc y permsrc en los proyectos IAF; src/agt, src/env, src/org y src/python en los proyectos JaCaMo). Claude Code solo la vuelve a enviar cuando cambia. Los agentes que pueden leer ficheros, como Claude Code con la herramienta Read, pueden abrir entonces el código del proyecto.

12.2 Los agentes de una instalación

Los agentes de cada usuario se guardan por usuario y por instalación en editor/config/users/<usuario>/agents.json, que solo puede leer su propietario porque puede contener claves de API. La primera vez que un usuario arranca el editor, el fichero se crea a partir de los agentes predeterminados de la instalación, editor/config/agents-default.json:

Agente	Contexto
Claude	Automático: el perfil activo del editor, que se lee en cada mensaje.

Agente	Contexto
Asistente JaCaMo	Siempre AgentSpeak (Jason/JaCaMo: .asl, artefactos CArtAgO, organizaciones Moise, .jcm).

Son los agentes de la distribución AgentSpeak (la distribución completa tiene además el *Asistente IAF*). Los dos usan Claude Code, con el modelo sonnet, esfuerzo bajo y las herramientas Read, Glob y Grep (pueden leer los ficheros del proyecto, pero no cambiarlos). Si Claude Code no está instalado pero está definida la variable de entorno ANTHROPIC_API_KEY, usan en su lugar la API de Anthropic. Quien despliega el IADK puede editar agents-default.json antes de desplegarlo; editor/config/README.txt describe su formato. En una instalación compartida, los usuarios necesitan permiso de escritura en editor/config/users para guardar sus agentes (chmod 1777 editor/config/users).

12.3 Gestionar los agentes

Preferences > *Agentes de IA...*, o el botón *Configurar...* de la pestaña de un agente, abre el gestor de agentes (figura 21). Permite añadir (*Añadir...*), editar (*Editar...*), duplicar, reordenar (*Subir*, *Bajar*) y eliminar agentes; *Guardar* guarda los cambios y las pestañas cambian en el momento, sin reiniciar el editor. *Predeterminados...* añade los agentes predeterminados de la instalación a los que ya tiene el usuario (figura 22).



Figura 21: El gestor de agentes (Preferences > Agentes de IA...)

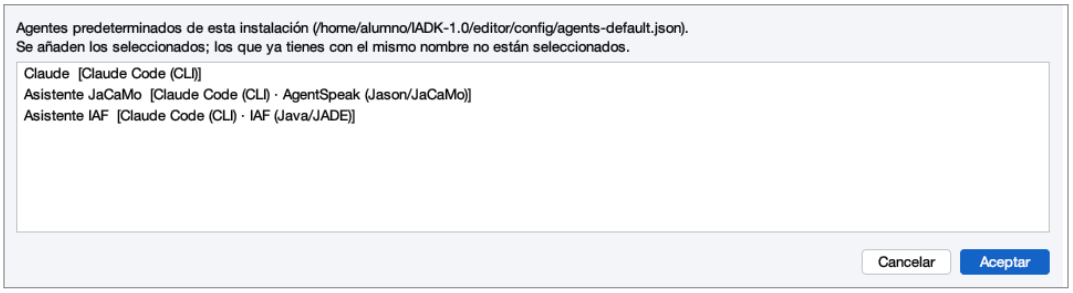


Figura 22: Añadir los agentes predeterminados de la instalación (Predeterminados...)

El diálogo de edición de un agente solo muestra los campos de su tipo (figura 23). Los campos comunes a todos los tipos son:

Campo	Significado
Nombre (pestaña)	Nombre del agente y de su pestaña.
Tipo	Claude Code (CLI), Agente de línea de órdenes (cualquier programa), API de Anthropic o API compatible con OpenAI.
Modelo	Modelo del agente. Depende del tipo: un alias como sonnet u opus para Claude Code; el nombre del modelo para las API.
Contexto	Con qué se le dice al agente que trabaja el usuario: Automático (el perfil activo), INGENIAS, IAF (Java/JADE) o AgentSpeak (Jason/JaCaMo).

Campo	Significado
Enviar como contexto la especificación abierta	Envía la especificación abierta y las carpetas de su proyecto.
Instrucciones adicionales	Texto que se añade a las instrucciones del agente (por ejemplo "Responde en inglés" o "Sigue las convenciones de la asignatura").

12.4 Paso a paso: Claude Code

Claude Code es el agente predeterminado. El editor lo ejecuta en modo no interactivo para cada mensaje y Claude Code mantiene la conversación, así que cada vez solo se envía el texto nuevo.

1. Instala Claude Code (https://claude.com/claude-code), por ejemplo con su instalador (`curl -fsSL https://claude.ai/install.sh | bash` en macOS y Linux) o con npm (`npm install -g @anthropic-ai/claude-code`). También sirve el binario que incluye la extensión de Claude Code para VS Code.
2. Abre un terminal, ejecuta `claude` una vez e inicia sesión con una cuenta de Claude (o configura una clave de API). Ciérralo con /exit.
3. Arranca el editor. La pestaña *Claude* ya usa Claude Code: su línea de estado muestra el modelo y el esfuerzo. Si dice "NO ENCONTRADO", el editor no ha encontrado el programa.
4. Si no lo encuentra, abre *Configurar...*, edita el agente y escribe en *Programa* la ruta completa de claude (el resultado de `which claude` en macOS y Linux, o de `where claude` en Windows). El editor busca claude en el PATH, en el PATH de un shell de inicio de sesión, en ~/.local/bin, /opt/homebrew/bin, /usr/local/bin, ~/.npm-global/bin y en la extensión de VS Code.
5. Si quieres, cambia los demás campos (figura 23): *Modelo* (sonnet, opus, haiku o el nombre completo de un modelo; vacío, el de la cuenta), *Esfuerzo* (low, medium, high, xhigh, max), *Herramientas* (vacío: ninguna, el agente solo conversa; "default": todas, incluidas editar ficheros y ejecutar órdenes; o una lista como Read,Glob,Grep o Read,Glob,Grep,Edit), *Carpeta de trabajo* (vacía: la carpeta del proyecto de la especificación abierta) y *Argumentos* (argumentos adicionales de claude).
6. *No cargar mis servidores MCP* (marcado por defecto) hace que arranque más rápido, sin los servidores MCP de la configuración del usuario.
7. Pulsa *Aceptar* y *Guardar*, y escribe una petición en la pestaña, por ejemplo: *"Revisa los objetivos y tareas de la especificación abierta: ¿hay alguna tarea que no satisfaga ningún objetivo?"*.

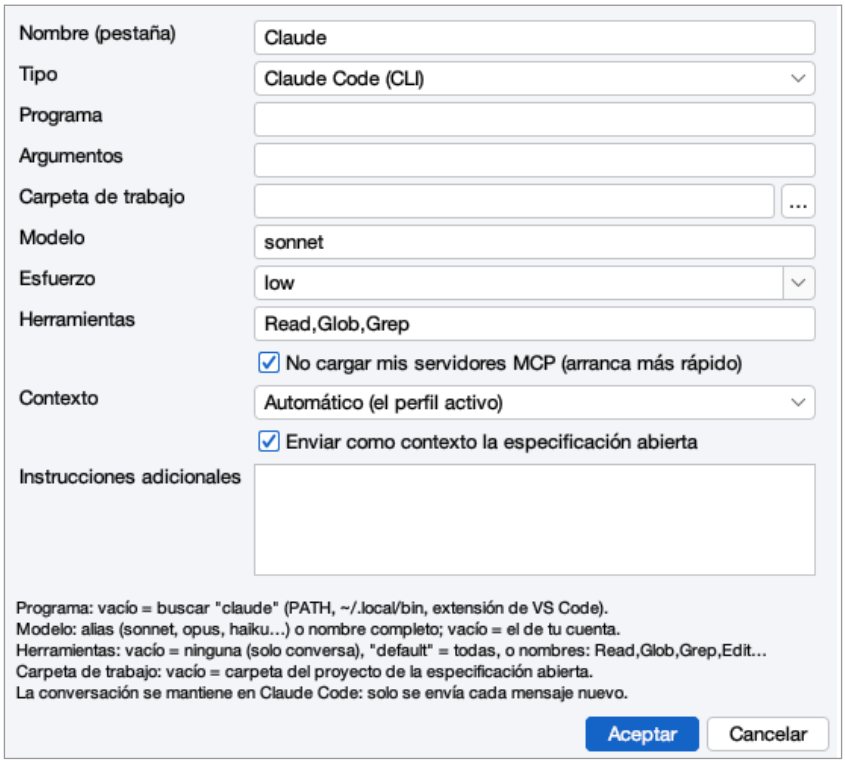


Figura 23: Configuración de un agente Claude Code

> Con las herramientas Read, Glob y Grep el agente puede leer el proyecto pero no cambiarlo. Dale Edit (y Write) solo cuando quieras que cambie el código, y revisa sus cambios antes de volver a generar el código: el código de los componentes de código está en el modelo, y los generadores reescriben gensrc y los ficheros .asl.

12.5 Paso a paso: otros agentes de línea de órdenes (Codex, Gemini, Ollama...)

El tipo *Agente de línea de órdenes* ejecuta cualquier programa para cada mensaje: la petición (con las instrucciones, la especificación abierta y los mensajes anteriores de la conversación) sustituye a {prompt} en los *Argumentos*; si los argumentos no tienen {prompt}, la petición se escribe en la entrada estándar del programa. Lo que imprime el programa es la respuesta. La carpeta de trabajo es la del campo *Carpeta de trabajo* o, si está vacío, la del proyecto.

Se recomienda enviar la petición por la entrada estándar: la especificación puede ser larga, y la longitud de la línea de órdenes es limitada (sobre todo en Windows). Las configuraciones siguientes se han preparado con las versiones de estos programas en el momento de escribir esta guía; comprueba sus opciones con `<programa> --help`, porque cambian de una versión a otra.

OpenAI Codex CLI (figura 24):

1. Instálalo (`npm install -g @openai/codex`) e inicia sesión una vez ejecutando `codex` en un terminal.
2. En el gestor de agentes, *Añadir...*: *Nombre* Codex, *Tipo* Agente de línea de órdenes, *Programa* codex.
3. *Argumentos*: `exec --skip-git-repo-check --sandbox read-only -` (exec atiende una sola petición; el - final la lee de la entrada estándar; read-only le deja leer el proyecto pero no cambiarlo; --skip-git-repo-check admite proyectos que no son repositorios git).
4. Elige el *Contexto* (Automático, o el de tu perfil: IAF (Java/JADE) o AgentSpeak (Jason/JaCaMo)), *Aceptar* y *Guardar*.

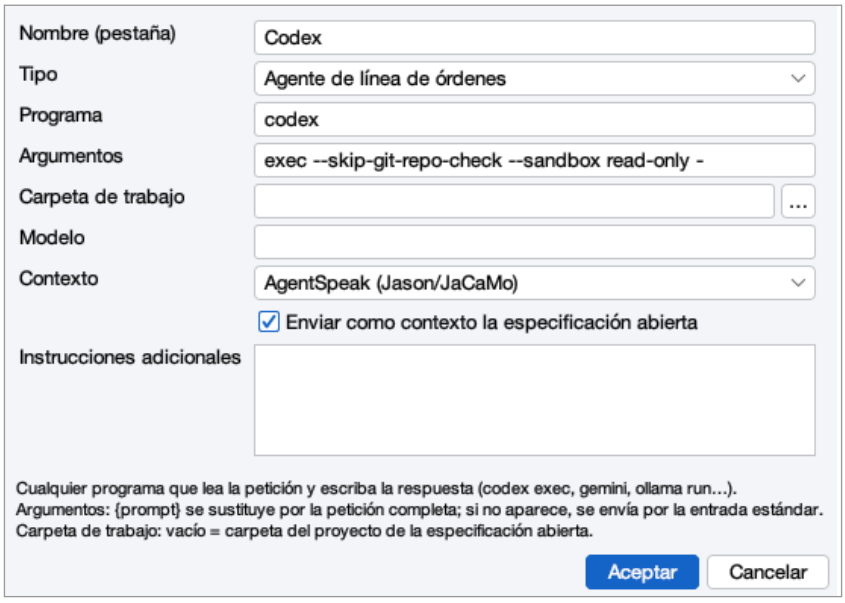


Figura 24: Configuración de un agente de línea de órdenes (Codex CLI)

Google Gemini CLI:

1. Instálalo (`npm install -g @google/gemini-cli`) e inicia sesión una vez ejecutando `gemini`.
2. *Nombre* Gemini, *Tipo* Agente de línea de órdenes, *Programa* gemini.
3. *Argumentos*: `-p "Responde a la petición recibida por la entrada estándar"`. Con -p, Gemini responde una vez y termina; el texto de -p se añade a la petición leída de la entrada estándar.

Ollama (un modelo local, desde la línea de órdenes):

1. Instala Ollama (https://ollama.com) y descarga un modelo: `ollama pull qwen2.5-coder`.
2. *Nombre* Ollama CLI, *Tipo* Agente de línea de órdenes, *Programa* ollama, *Argumentos* `run qwen2.5-coder`.

Para Ollama es preferible la API compatible con OpenAI de la sección siguiente: muestra la respuesta a medida que se genera y mantiene la conversación como mensajes.

Cualquier otro programa o script. Un script que lea la entrada estándar e imprima la respuesta se usa del mismo modo, por ejemplo para llamar a un modelo de un servidor de la empresa.

12.6 Paso a paso: API de Anthropic

El tipo *API de Anthropic* llama directamente a la Messages API, sin Claude Code (figura 25).

1. Crea una clave de API en la consola de Anthropic (https://console.anthropic.com).
2. Defínela en una variable de entorno antes de arrancar el editor, por ejemplo ANTHROPIC_API_KEY. En macOS y Linux, añade `export ANTHROPIC_API_KEY=sk-ant-...` a ~/.zshrc o ~/.bashrc; en Windows, `setx ANTHROPIC_API_KEY sk-ant-...` y abre un terminal nuevo. Arranca el editor desde un terminal con ese entorno (con doble clic en IADK.command o IADK.bat puede no leerse la configuración del shell).
3. *Añadir...*: *Nombre* Claude API, *Tipo* API de Anthropic, *Variable con la clave* ANTHROPIC_API_KEY.
4. *Modelo*: vacío usa el modelo predeterminado del editor (claude-sonnet-5); también se puede escribir el nombre de otro modelo de la API. *URL*: vacía usa https://api.anthropic.com/v1/messages. *Máximo de tokens*: la longitud máxima de las respuestas (8192).
5. *Aceptar*, *Guardar*, y escribe una petición en la pestaña nueva.

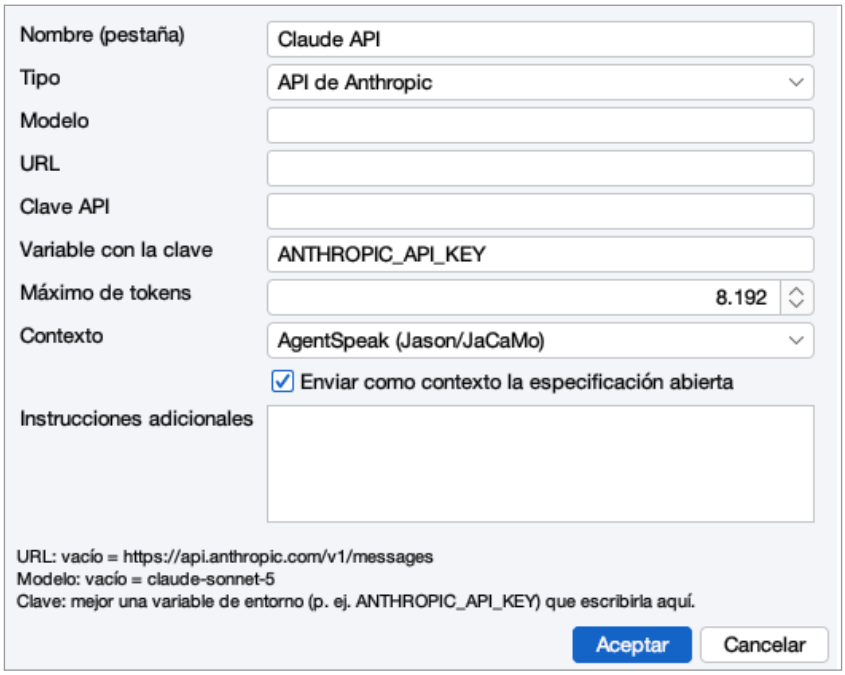


Figura 25: Configuración de un agente de la API de Anthropic

> La clave también se puede escribir en el campo *Clave API*, pero entonces se guarda en agents.json. Una variable de entorno es más segura: nunca se guarda en la instalación, y agents.json se puede copiar a otras instalaciones sin exponer la clave.

12.7 Paso a paso: API compatibles con OpenAI (OpenAI, Ollama, LM Studio)

El tipo *API compatible con OpenAI* usa la API de chat completions, que ofrecen OpenAI y muchos servidores locales (figura 26). El *Modelo* es obligatorio.

Ollama (modelo local, sin clave):

1. Instala Ollama (https://ollama.com), arráncalo y descarga un modelo, por ejemplo `ollama pull qwen2.5-coder` (o llama3.1).
2. *Añadir...*: *Nombre* Ollama, *Tipo* API compatible con OpenAI.
3. *URL*: `http://localhost:11434/v1/chat/completions`; *Modelo*: qwen2.5-coder; deja vacíos los campos de la clave.
4. *Aceptar*, *Guardar*. El modelo se ejecuta en el ordenador: la especificación no se envía a ningún servicio externo.

LM Studio: arranca su servidor local (pestaña Developer) con un modelo cargado, y usa la URL `http://localhost:1234/v1/chat/completions` y el nombre del modelo.

OpenAI: crea una clave de API (https://platform.openai.com), defínela en la variable de entorno OPENAI_API_KEY, deja vacía la *URL* (https://api.openai.com/v1/chat/completions), escribe el modelo y pon OPENAI_API_KEY en *Variable con la clave*.

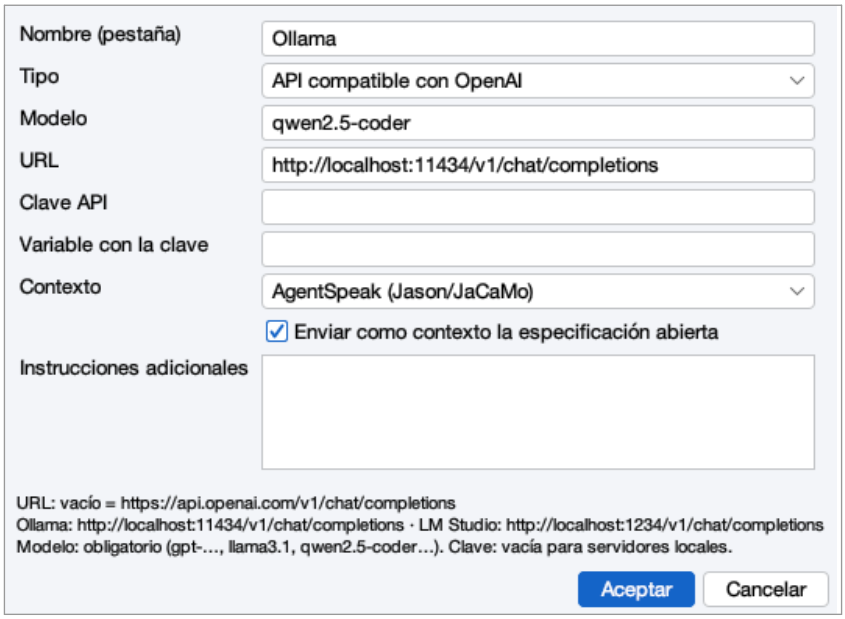


Figura 26: Configuración de un agente compatible con OpenAI (Ollama)

12.8 Agentes para un perfil

El *Contexto* de un agente decide con qué se le dice que trabaja el usuario. Con *Automático* el agente sigue al perfil elegido en el editor; un agente con un contexto fijo (por ejemplo, el *Asistente JaCaMo*) habla siempre de su plataforma, sea cual sea el perfil activo. Se pueden tener a la vez varios agentes de distintos tipos y contextos, y el usuario elige la pestaña que quiere usar: por ejemplo, Claude Code para el código y un modelo local para las preguntas que no deben salir del ordenador.

12.9 Peticiones útiles

Guarda la especificación antes de preguntar: el agente recibe el fichero guardado.

- "Explica las reglas de greeter.asl y su relación con los planes del modelo."
- "¿Por qué no se satisface nunca el objetivo reach_target?" (pegando en la petición la salida de la aplicación).
- "Escribe el cuerpo de la tarea step para que pare cuando el contador llegue al valor de la creencia target."
- "Revisa la organización Moise: ¿hay algún rol con una misión cuyos objetivos no pueda alcanzar ningún agente?"
- "Escribe la operación reset del artefacto Counter y dime cómo añadirla al modelo."

El código de los planes y las tareas está en el modelo: si el agente cambia el .asl (con la herramienta Edit), sube los cambios con el CodeUploader o el Code synchronizer antes de volver a generar el código.

12.10 Problemas frecuentes con los agentes de IA

Problema	Solución
"Claude Code ... NO ENCONTRADO" en la línea de estado	Instala Claude Code o escribe la ruta completa de claude en Programa (sección 12.4).
"SIN CLAVE API" en la línea de estado	La variable de la clave no está definida en el entorno del editor: arranca el editor desde un terminal donde esté definida, o revisa el nombre en Variable con la clave.
HTTP 401 o 403	La clave es incorrecta o no tiene permisos para el modelo.
HTTP 404 con un servidor compatible con OpenAI	URL o nombre del modelo incorrectos (con Ollama, `ollama list` muestra los modelos).
Un agente de línea de órdenes termina con un código de error	Ejecuta la misma orden en un terminal, en la carpeta del proyecto, para ver su mensaje (por ejemplo, que no ha iniciado sesión).
El agente no ve los últimos cambios	Guarda la especificación: se envía el fichero guardado.
No se pueden guardar los cambios (Guardar)	El usuario no tiene permiso de escritura en editor/config/users.

13. PROBLEMAS FRECUENTES

Problema	Qué comprobar
`./gradlew run` falla la primera vez	Necesita Internet para descargar Gradle y JaCaMo; detrás de un proxy, configúralo en ~/.gradle/gradle.properties.
El agente no hace nada	Sus objetivos iniciales: un plan que no satisface ningún objetivo solo es una intención inicial si sus reglas se disparan con su nombre. Mira la sección "Initial goals" del .asl.
No hay plan aplicable para un objetivo	Falta el plan del objetivo, o no se cumple el contexto de sus reglas. Revisa las GTSatisfies de los planes y las tareas.
El objetivo no termina nunca	Sus SatisfactionBeliefs no se cumplen nunca, o ninguna regla las añade. Revisa las creencias con el inspector de la mente.
"artifact_name only works if either wid or wsp is also informed"	Una acción de organización sin el workspace de la organización: el generador añade wsp(org); haz lo mismo en el código escrito a mano.
No se encuentra una operación de un artefacto	El agente no hace focus en el artefacto: añade la EPerceives del agente a la EnvironmentApplication.
Falla leaveMission o leaveRole	Normas de Moise: no se puede abandonar una misión hasta alcanzar sus objetivos, ni un rol que el grupo necesita para sus esquemas.
Mis reglas han desaparecido del .asl	Estaban fuera de las marcas, o no se subieron antes de generar. Usa el CodeUploader (AgentSpeak) antes de generar.
Falla un artefacto Python	Python 3 debe estar instalado (o JACAMO_PYTHON definida); el error del módulo aparece en el terminal.
El generador escribe el proyecto en otra carpeta	La propiedad JaCaMo project folder de Project > Properties: déjala vacía para usar la carpeta de la especificación.
Un elemento del modelo no se genera	La pestaña Logs dice por qué (relaciones que el perfil no usa, elementos sin objetivo...) y cómo modelarlo.